Introducing kbretrieveR: Fetch insights from your AWS Knowledge Base in R

R-native retrieval and grounded chat with AWS Bedrock Knowledge Bases

AUTHOR
Lazaro Alvarez

PUBLISHED
October 9, 2025

Fetching insights directly from your AWS Knowledge Bases

If you've ever tried to connect an R workflow to a large-scale knowledge system like **AWS Bedrock**, you've probably noticed one thing — **most retrieval and RAG tooling is built for Python.**

While packages like `LangChain` exploded in that ecosystem, R users were often left without a clean, native way to query their organization's private Knowledge Bases.

`kbretrieveR` is an R package that lets you work directly with AWS Knowledge Bases. If your project has a Knowledge Base with project details or coding best practices, you can connect it to `ellmer` and feed that context into your R session. From there, you can generate parameterized Markdown or Quarto documents that automatically incorporate the KB as context.

The package is more than just a chat client—it's a building block. Its real value comes when you use it in parameterized reports or AI agents. Instead of hard-coding prompts or constantly updating them as information changes, `kbretrieveR` lets your AI workflows dynamically retrieve the latest knowledge base context, ensuring your R agents can generate outputs grounded in up-to-date, project-specific information.

- Official Github https://github.com/MDRCNY/kbretrieveR

Why It Matters

In research and analytics organizations, knowledge isn't just in datasets — it's in PDFs, Word docs, policy manuals, and archived reports scattered across SharePoint and S3. AWS Bedrock now provides a secure way to index all that institutional knowledge, but analysts still need a bridge to use it in day-to-day work.

That's where `kbretrieveR` comes in. It brings the retrieval layer directly into R — no Python wrappers, no external APIs, no leaving your workflow. You can ask questions, summarize policies, or automatically cite knowledge base documents within Quarto reports or dashboards.

A Package Built for Real Analysts

`kbretrieveR` was designed for the environments where analysts actually work — often inside `controlled AWS environments`.

Under the hood, it uses secure HTTPS requests to the AWS Bedrock API, following the same SigV4 authentication process used by the official SDKs. This means it integrates cleanly with your existing AWS credentials, IAM roles, and permissions — making it both secure and compliant by default.

Whether you're a data scientist writing reproducible research or an analyst assembling a compliance report, `kbretrieveR` makes it possible to:

- 🔍 Search enterprise knowledge bases directly from R
- 📄 Retrieve relevant documents and metadata for citation
- 💬 Chat with Bedrock LLMs like Claude or Titan using grounded knowledge
- 🧩 Integrate AI-powered retrieval into reproducible pipelines and Quarto reports

Installation

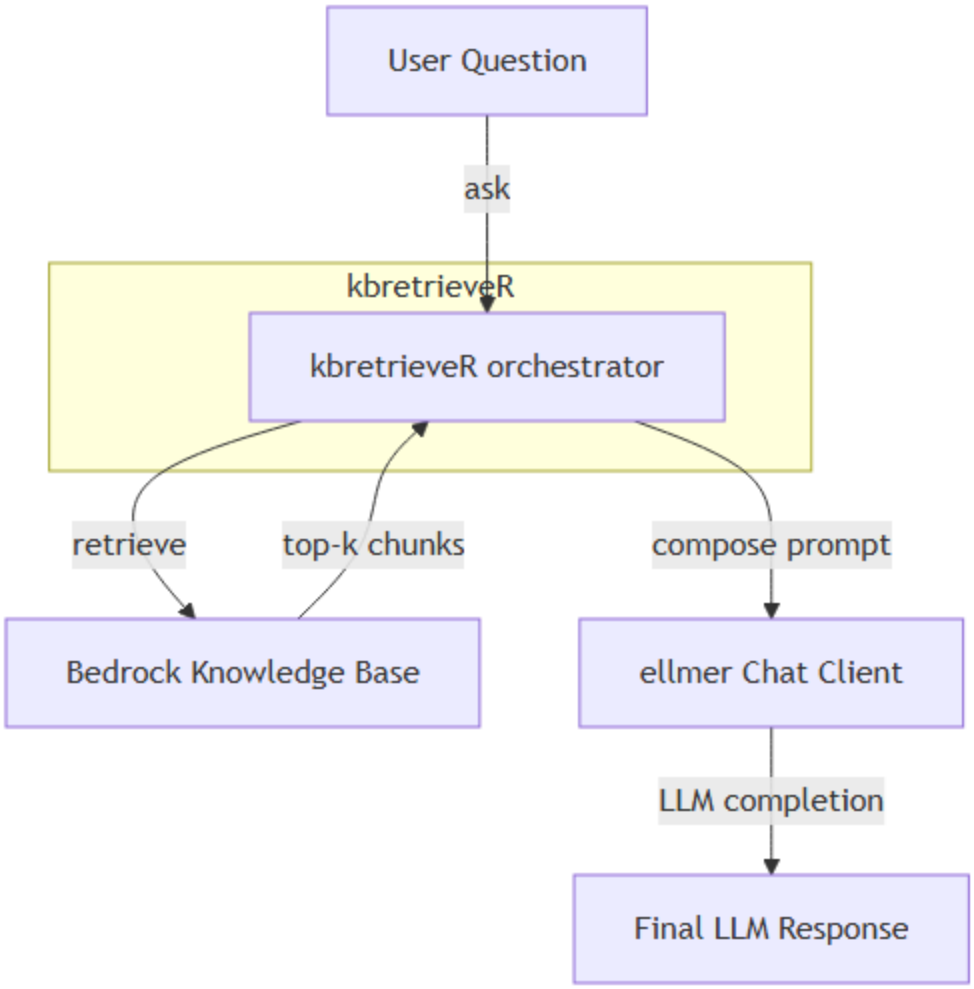
```
# from devtools / remotes
remotes::install_github("MDRCNY/kbretrieveR")
```

Quick Start

```
library(kbretrieveR)

# Create a client (replace with your KB ID & region)
client <- KBClient$new(
  kb_id = "1234ABCE", # OR Sys.getenv("AWS_KB_ID")
  region = "us-gov-west-1", # OR Sys.getenv("AWS_REGION")
  chat_client = ellmer::chat_aws_bedrock(
    model = "anthropic.claude-3-5-sonnet-20240620-v1:0"
  )
)

# Retrieve docs - will print tibble table of retrieved context chunks from AWS KB
client$retrieve("Tell me about my companies new projects?")

# Chat with context injected
client$chat("Summarize the companies new projects in 3 bullet points.")

# Limit sources to only those with highest relevance and append the sources
client$retrieve("Tell me about my companies new projects?", number_of_results=3)
client$chat("Summarize in 3 bullet points", number_of_results=3, append_sources = TRUE)
```

⚙️ How It Works

When you ask a question, kbretreiveR acts as an orchestrator between your R session, the AWS Bedrock Knowledge Base, and an LLM like Claude or Titan. It first retrieves the top-k relevant document chunks from the Knowledge Base, then composes a grounded prompt combining your question and those retrieved texts. That prompt is sent through the ellmer chat client to the LLM, which generates a final, context-aware answer — ensuring every response is both relevant and traceable to real enterprise knowledge.



Example: use KB results in a reproducible report

You can store defaults in your `.Renviron` for convenience similar to `ellmer`:

```
AWS_ACCESS_KEY_ID = "  "
AWS_SECRET_ACCESS_KEY = "  "
AWS_SESSION_TOKEN = "  "
AWS_REGION = "us-east-1"

AWS_KB_ID="1234ABCE"
```

This example demonstrates a typical analyst workflow: 1. Search a Knowledge Base for relevant documents 2. Retrieve full document(s) for the top hits

3. Extract short text snippets and join them to a data frame
4. Render a small Quarto report section that includes the search table and a quoted snippet with citation

```
> library(kbretrieveR)
> client <- KBClient$new(
+   kb_id = Sys.getenv("AWS_KB_ID"),
+   region = Sys.getenv("AWS_REGION"),
+   chat_client = ellmer::chat_aws_bedrock("anthropic.claude-3-5-sonnet-20240620-v1:0")
+ )
Using model = "anthropic.claude-3-5-sonnet-20240620-v1:0".
> # Defualt number of results = 5
> client$chat("Tell me about kbretrieveR")
Using region=us-gov-west-1 (host=bedrock-agent-runtime.us-gov-west-1.amazonaws.com). Signing with
Sending prompt to chat client (prompt length: 4885 chars).
Based on the provided context, here's what I can tell you about kbretrieveR:

1. kbretrieveR is an R package that provides a thin interface for working with AWS
Bedrock Knowledge Bases.

2. Its main functionality is to retrieve relevant chunks of information from an AWS
Bedrock Knowledge Base and merge them into prompts that can be passed to chat
clients (such as ellmer::chat_aws_bedrock).

3. The package is designed to be used as a building block in parameterized reports
or AI agents, allowing for dynamic retrieval of up-to-date, project-specific
information from knowledge bases.

4. It can be installed from GitHub using devtools or remotes:
   ~~~R
   remotes::install_github("MDRCNY/kbretrieveR")
   ~~~

5. The package uses a client-based approach. Users create a KBClient object with
their Knowledge Base ID, region, and preferred chat client.

6. Example usage:
   ~~~R
   library(kbretrieveR)

   client <- KBClient$new(
     kb_id = "1234ABCE",
     region = "us-east-1",
     chat_client = ellmer::chat_aws_bedrock(
       model = "anthropic.claude-3-5-sonnet-20240620-v1:0"
     )
   )

   docs <- client$retrieve("Tell me about my companies new projects?")
   reply <- client$chat("Summarize in 3 bullet points.")
   ~~~
```

```
7. The package allows for configuration through environment variables, similar to
the ellmer package.

8. kbretrieveR is particularly useful for generating parameterized Markdown or
Quarto documents that automatically incorporate knowledge base context.

9. The GitHub repository for kbretrieveR is maintained by Lazaro Alvarez.

This package seems to be a useful tool for R users who want to integrate AWS
Bedrock Knowledge Bases into their workflows, especially for AI-assisted document
generation and information retrieval.
```

```
> client$retrieve("Tell me about kbretrieveR")
Using region=us-east-1 (host=bedrock-agent-runtime.us-east-1.amazonaws.com). Signing with key=ASI
\# A tibble: 5 × 7
  score uri                                text  source_uri                     chunk_id   da
  <dbl> <chr>                              <chr> <chr>                          <chr>      <c
1 0.742 s3://bedrock-docs-rag-materials/f1b2 "kbretrieveR connects R users to AWS…" s3://docs… 2%
2 0.618 s3://bedrock-docs-rag-materials/a83d "Built for secure, offline-friendly …" s3://docs… 4%
3 0.557 s3://bedrock-docs-rag-materials/c92e "Each call is AWS SigV4 signed and s…" s3://docs… 6%
4 0.496 s3://bedrock-docs-rag-materials/d4ab "Users can retrieve or chat with kno…" s3://docs… 3%
5 0.463 s3://bedrock-docs-rag-materials/ef77 "# Example\nkb <- KBClient$new(kb_id…" s3://docs… 1%
```

🔧 KBClient Configuration & Streaming

`KBClient()` is the core interface in `kbretrieveR` for interacting with an AWS Bedrock Knowledge Base and optionally calling a chat model (e.g. via `ellmer`). It stores convenient defaults (KB ID, region, chat client, and prompt limits) and can optionally stream responses when supported by the underlying model client.

🧱 Constructor

```
client <- KBClient$new(
  kb_id = Sys.getenv("AWS_KB_ID"),                # Knowledge Base ID
  region = Sys.getenv("AWS_REGION", "us-east-1"),  # AWS region
  chat_client = ellmer::chat_aws_bedrock(
    model = "anthropic.claude-3-5-sonnet-20240620-v1:0"
  ),
  default_number_of_results = 5,    # Default KB retrieve size
  default_max_snippets = 5,         # Default number of context snippets injected into the prompt
  default_snippet_chars = 1500,     # Max characters per snippet
  include_metadata = TRUE,          # Include metadata (source URIs, chunk IDs) in the prompt
  verbose = TRUE                    # Print progress messages
)
```

Chat Configuration Options

Method	Description
`$retrieve(question, number_of_results = 5)`	Retrieve context snippets from the Knowledge Base. Returns a tibble with text, URIs, and scores.
`$chat(question, ...)`	Retrieve KB context, build a prompt, and query the chat client. Returns the assistant reply.
`$inspect(question)`	Pretty-print the retrieved snippets and their metadata for inspection.
`$set_chat_client(client)`	Replace the chat client (e.g., swap between Bedrock models).
`$format_citation(row)`	Format a single KB row into a markdown citation link.
`$as_list()`	Return all internal defaults and configuration values.

Example in R Code:

```
client$chat(
  question,               # user input text
  kb_id = NULL,           # override KB ID
  chat_client = NULL,     # override chat client
  number_of_results = 5,  # KB context size
  max_snippets = 5,       # number of snippets to inject
  snippet_chars = 1500,   # characters per snippet
  append_sources = FALSE, # append "Sources" list to reply
  return_raw = FALSE,     # return full response + prompt + parsed KB
  verbose = TRUE,         # show progress
  stream = FALSE,         # ⚡ optional streaming mode (default = FALSE)
  stream_fn = NULL        # optional custom streaming wrapper
)
```

> Note: Streaming is off by default to preserve backwards-compatible, synchronous behavior. You can enable it per-call with stream = TRUE, or globally via an R option:

```
options(kbretrieveR.stream_default = TRUE)
```

Conclusion

`kbretrieveR` is meant to feel like R: pipe-friendly, reproducible, and comfortable inside RStudio/Posit—while quietly doing the hard work of securely talking to AWS Bedrock behind the scenes. With a few lines of code, you can search your organization's Knowledge Bases, pull in the most relevant passages, and ground your LLM prompts so your reports, dashboards, and agents stay accurate and traceable.